

Intelligent OCR for Scanned Documents

French — English — Arabic

Technical note: rationale for the OCR engine selection

July 2026

Contents

1 Context and Objectives **2**

2 Functional Scope **2**

3 Overall Architecture **3**

4 Detailed Technical Architecture **4**
- 4.1 Image preprocessing . . . 4
- 4.2 Local inference service . . . 4
- 4.3 Memory constraint and GPU residency . . . 4
- 4.4 The `-instruct` suffix is mandatory . . . 4
- 4.5 Context-window tuning . . . 4
- 4.6 Multi-page PDF handling . . . 4
- 4.7 Prompt design . . . 4
- 4.8 Cloud path (optional) . . . 5

5 Recognition Modes **5**

6 Quality Control and Validation **5**
- 6.1 Measurement protocol . . . 5
- 6.2 The dominant failure mode is omission . . . 6
- 6.3 Recall verification (two-pass) . . . 6
- 6.4 Two-model completeness comparison . . . 6
- 6.5 Format validation without ground truth . . . 6
- 6.6 Known limitations . . . 7

7 Technology Choices **7**
- 7.1 Why a VLM rather than classical OCR . . . 7
- 7.2 Why the Qwen3-VL family . . . 7
- 7.3 Why the 4B size, and why the *instruct* variant . . . 7
- 7.4 Why local (Ollama) is the default engine . . . 8
- 7.5 Why Groq is kept as an option . . . 8
- 7.6 Why this delivery stack . . . 8

8 Delivered Interfaces **8**

9 Project Structure **8**

10 Performance **9**

11 Development Planning **10**

12 Conclusion **10**

1 Context and Objectives

The project addresses the need to digitise and automatically process **scanned** documents: administrative forms, reports and handwritten material, written in **French**, **English** or **Arabic**.

Classical OCR engines (Tesseract, for example) detect *character shapes* with no understanding of context, layout or language. They degrade sharply on noisy scans, mixed scripts, tables and handwriting — precisely the cases encountered here.

By contrast, a **Vision-Language Model** (VLM) reads an image and produces text within a *single* neural network. It is this joint understanding of image and language that makes difficult documents tractable.

Objectives.

- Faithfully transcribe multilingual scanned documents (FR / EN / AR, including right-to-left script).
- Preserve the document **structure** (headings, tables, checkboxes), not merely the raw text.
- Guarantee **confidentiality**: the reference document used for testing is a strictly confidential safeguarding form containing names and medical details. No document may leave the machine.
- Run on the hardware actually available: a laptop fitted with an **NVIDIA RTX 4050 (6 GB VRAM)**.
- Incur no recurring cost: no per-page billing, no quota.
- Deliver the capability as a usable product, not only as a script: a multi-user web platform with authentication and a persistent per-user document history.

2 Functional Scope

In scope.

- **Inputs**: images (PNG / JPG / WEBP) and scanned PDFs (multi-page, rasterised locally).
- **Languages**: French, English, Arabic — including mixed FR/AR documents and non-Latin right-to-left scripts.
- **Content handled**: dense text, tables, structured forms, checkboxes and tick-boxes, handwriting.
- **Output**: structured text in **Markdown**, preserving the hierarchy and tables of the source document, with checkbox states reported explicitly.
- **Execution modes**: local engine by default, second opinion (quality assurance) on a larger model, optional cloud engine.
- **Quality control without ground truth**: recall verification, two-model completeness comparison, and format validation of constrained fields.
- **Interfaces**: a full-stack web platform (primary deliverable), plus a Streamlit prototype and a command-line interface over the same OCR core.
- Full **offline operation**.

Out of scope.

- Cloud processing as the default: ruled out for data-governance reasons.
- Business validation of the extracted content: the system transcribes; it neither interprets nor validates medical information.
- High-throughput batch processing: the target is the individual document, at roughly 30 seconds per page.
- Public deployment: there is no user signup flow by design; accounts are seeded or created directly in the database.

3 Overall Architecture

The architecture rests on one guiding principle: **local-first**. The nominal engine runs entirely on the user's machine; the cloud is only an optional alternative.

Processing chain.

1. **Ingestion**: loading the image or scan. PDFs are rasterised locally, one image per page.
2. **Preparation**: image normalisation (denoising, deskew, contrast) and construction of the transcription prompt.
3. **VLM inference**: reading the image and generating the text with the vision-language model, served locally.
4. **Structuring**: Markdown formatting of the output (headings, tables, lists), language detection and right-to-left handling.
5. **Quality control (optional)**: recall verification pass, second opinion on a larger model, format validation of constrained fields.
6. **Delivery**: hand-off of the transcribed document — displayed, downloadable, and stored in the user's history.

Engines involved. Four engines sit behind a single abstraction, so any of them can be swapped without touching the processing chain.

Role	Model	Where it runs
Default engine	Qwen3-VL 4B (instruct) — `qwen3-vl:4b-instruct`	Local, via Ollama, on the GPU
Second opinion (QA)	Qwen3-VL 8B (instruct) — `qwen3-vl:8b-instruct`	Local, via Ollama
Classical baseline	Tesseract (`fra+eng+ara`)	Local, optional — benchmarking and hybrid mode
Optional cloud	Llama 4 Scout (vision)	Groq API

Table 1: Division of roles between the engines

4 Detailed Technical Architecture

4.1 Image preprocessing

Preprocessing runs on **OpenCV**, with a Pillow-only fallback when OpenCV is unavailable: grayscale conversion, non-local-means denoising, deskew (estimation of the dominant text angle, applied only between 0.3° and 15° to avoid acting on a misdetection), and CLAHE contrast enhancement. Adaptive binarisation is applied *only* on the Tesseract path: the vision model performs measurably better on contrast-enhanced grayscale than on a hard-thresholded image.

Images are normalised to the model's useful resolution — long edge capped at 2576 px, small scans upscaled — and encoded as JPEG to keep the payload within the limits of the vision APIs.

4.2 Local inference service

The models are served by **Ollama**, which exposes a local API and manages loading the weights onto the GPU. No outbound network call is required for nominal operation.

4.3 Memory constraint and GPU residency

The RTX 4050 provides **6 GB of VRAM**. The 4B model occupies about **3.3 GB** and therefore stays *fully resident* on the GPU. Larger models spill over to the CPU, which severely degrades response times.

4.4 The `-instruct` suffix is mandatory

A decisive technical point: the base `qwen3-vl` tags run in *thinking* mode. They spend the entire token budget on internal reasoning and return an **empty transcription**. Only the `-instruct` variant transcribes the image content directly. The engine detects this failure mode explicitly and reports it rather than returning an empty result.

4.5 Context-window tuning

Setting the `num_ctx` parameter to **8192 tripled the speed** compared with the naive initial setting, bringing processing down to roughly **30 seconds per page** on the target hardware. The value is a genuine optimum rather than a maximum: each page is a separate call, so the window only needs to hold one page (roughly 4300 image tokens plus the transcription). Raising it further wastes VRAM on the KV cache and forces model layers onto the CPU — measured about three times slower per page, for no accuracy gain.

4.6 Multi-page PDF handling

Neither engine accepts PDF input natively. Pages are rasterised locally with **PyMuPDF** at 300 DPI (no external Poppler dependency) and transcribed one at a time, the results being joined with a page separator. The resolution matters: fine print — form placeholders, footnotes — sits right at the model's legibility threshold, and 200–250 DPI drops it where 300 DPI reads it reliably. Pages are processed sequentially: on a single GPU that is the fast path, since concurrent calls merely contend for the same VRAM.

4.7 Prompt design

Three system prompts drive the model, all sharing a strict "transcribe, never interpret" contract — no translation, no summarising, no grammar correction:

- **Extraction**: reproduce every character faithfully, including accents and Arabic diacritics; transcribe small print, footnotes, headers, stamps and form helper text; report the state of every checkbox next to its label; preserve reading order, right-to-left for Arabic passages; render structure as Markdown, tables as Markdown tables; mark genuinely illegible regions rather than guessing.
- **Proofreading**: used by the verification pass (Section 6.3).
- **Post-correction**: used by the hybrid mode to repair classical OCR output — character confusions, broken words, missing diacritics — without paraphrasing.

4.8 Cloud path (optional)

A much larger remote engine — **Llama 4 Scout** (17B, vision) via the **Groq** API — is integrated as a benchmark and an alternative for cases willing to trade confidentiality for maximum accuracy. It shares the same prompts and the same processing chain as the local engine, so local and cloud results are directly comparable. It is selected per request and requires an API key; the local engine remains the default and needs none.

5 Recognition Modes

Three modes run over the same chain, selectable per request in every interface:

1. **VLM vision** (recommended, default) — the preprocessed page is read directly by the vision-language model. Best on degraded scans, complex layouts, tables and Arabic script.
2. **Hybrid** — Tesseract produces a fast raw transcription, then the VLM repairs character confusions, broken words and accent errors without paraphrasing.
3. **Tesseract only** — the classical baseline, no VLM involved. Useful for benchmarking the VLM approach against a traditional one.

Modes 2 and 3 require the Tesseract binary with the `fra`, `eng` and `ara` language packs. Tesseract is auto-detected: when it is absent — the normal configuration — both modes are simply hidden from the interfaces, and only VLM vision is offered.

6 Quality Control and Validation

6.1 Measurement protocol

The comparison was run on the most demanding case in the corpus: a **dense handwritten form** containing tables and checkboxes. The chosen criterion is transcription **completeness**, supplemented by processing time. Where a ground truth is available, **CER** and **WER** (character and word error rate, Levenshtein-based, with Unicode NFC normalisation — required for consistent Arabic comparison) are computed instead.

Model	Completeness	Duration	Observation
Qwen3-VL 4B instruct	10/10	59 s	Fully resident on the GPU
Qwen3-VL 8B instruct	9/10	135 s	Spilled to CPU (6 GB VRAM insufficient)

Table 2: Results measured on the reference handwritten form

The result is counter-intuitive but unambiguous: on this machine the 4B model is both **faster and more complete** than the 8B. The choice is therefore not assumed, it is **measured**.

6.2 The dominant failure mode is omission

On a dense form, a vision model fails far more often by *skipping* text than by inventing it: faint print, marginal notes, table cells and checkboxes get silently dropped. Simply reading the page twice does not help — the model tends to skip the same things twice. Three independent tools address this from different angles.

6.3 Recall verification (two-pass)

The verification option runs a two-pass loop. The first pass is a normal transcription. The second pass gives the model *the same image plus its own draft*, under a prompt whose only job is to find what the draft missed and fix what it got wrong, while being forbidden to remove correct text, translate, reorder or summarise.

Handing the model its own output as a checklist turns an open-ended transcription task into a targeted search, which is what recovers the skipped spans. The cost is exactly **twice the model calls** — so about twice the time and twice the tokens, per page on a PDF.

6.4 Two-model completeness comparison

No OCR engine — local or cloud — can *guarantee* it never drops a word; accuracy is always a measured rate, not an absolute. The honest way to push towards completeness is to read each page with **two independent models** and review wherever they disagree: agreement means high confidence, disagreement marks a spot worth a human glance.

The comparison mode does exactly this, fully offline, using the 4B and 8B local models. It reports a word-level agreement percentage and every disagreement span, flagging text one model skipped and the other caught. In practice the two models have *different* blind spots — the 4B is stronger on some checkboxes and fine print, the 8B catches some headers the 4B drops — so the diff reliably surfaces real omissions that a single pass would hide.

6.5 Format validation without ground truth

Digits and proper nouns are where a vision model fails first: they carry no linguistic context to fall back on, so a misread `B` → `8` produces something perfectly plausible. On a form, those are exactly the fields that matter — a wrong postcode or phone number is worse than a wrong adjective.

Those fields are *format-constrained*, so a share of that error class is catchable **with no ground truth at all**. Three checks run over the transcription:

Check	What it catches	Example
Structural	The token violates the field's grammar	A UK postcode cannot hold a digit in its final two positions
Consistency	The same entity transcribed two ways	Two spellings of one postcode, both legal — so one is misread
Plausibility	Wrong length	A 9- or 10-digit UK phone number means a dropped digit

Table 3: Format checks applied to constrained fields

When exactly *one* OCR confusion substitution (`8` → `B`, `5` → `S`, `2` → `Z`, ...) repairs a malformed token, the fix is unambiguous and is printed as a suggestion; ambiguous repairs are reported without one. The rules are currently UK-specific, because that is what the test corpus contains.

This validates **form, not truth**: a well-formed but wrong value — a valid postcode for the wrong town — passes clean. A clean report means "no detectable format errors", never "no errors".

6.6 Known limitations

- **Hardware dependency.** Performance is tied to the presence of a GPU and its VRAM; on a machine without a GPU, processing times become prohibitive.
- **Scope of the measurement.** The figures above come from one reference document on one machine; they indicate a trend, not a universal guarantee.
- **Fidelity not guaranteed at 100%.** Given that the data is personal and medical, **human proofreading** of the transcription remains necessary. None of the three quality tools replaces it.
- **Arabic handwriting.** It remains the hardest case in the chain and accounts for most of the residual errors.
- **Third-party dependency.** The cloud path depends on a provider's model catalogue, which changes over time; the model identifier is therefore configurable rather than hard-coded.
- **Validation rules are locale-specific.** The format checks encode UK postcode and phone grammar and would need extending for other locales.

7 Technology Choices

7.1 Why a VLM rather than classical OCR

Classical OCR recognises glyphs; a VLM *understands* the page. That understanding of context, layout and language is what makes it possible to handle degraded scans, mixed scripts, tables and handwriting. Tesseract is nonetheless retained as an optional baseline, so the claim can be verified by measurement rather than asserted.

7.2 Why the Qwen3-VL family

- **Strong multilingual performance, Arabic included.** The project targets French, English and Arabic (right-to-left). Qwen models are trained heavily on multilingual and Arabic data, and rank among the best open VLMs on non-Latin scripts — confirmed on the mixed FR/AR documents in the corpus.
- **Built for document tasks.** They handle dense text, tables and structured forms, and produce structured **Markdown** output: exactly what the reference form required, with its tables and checkboxes.
- **Open weights, local execution.** Usable through Ollama with no licence fee.

7.3 Why the 4B size, and why the *instruct* variant

- **4B fits the hardware.** About 3.3 GB, meaning full residency within the 6 GB of VRAM on the RTX 4050.
- **Measured, not assumed.** 10/10 in 59 s for the 4B against 9/10 in 135 s for the 8B (see Section 6.1).
- **The `-instruct` tag is imperative.** Without it, the model reasons instead of transcribing and returns an empty output.

7.4 Why local (Ollama) is the default engine

- **Confidentiality.** The test document is a strictly confidential safeguarding form with names and medical details. Local processing means *no document ever leaves the machine*: the decisive factor for data governance.
- **No cost, no limits, offline.** No per-page API fees, no rate limits, no need for an internet connection.
- **Fast enough on real hardware.** Roughly 30 seconds per page after tuning the context window.

7.5 Why Groq is kept as an option

This remote engine serves as a **benchmark** and an **alternative** towards a substantially larger model, for cases that would accept trading confidentiality for maximum accuracy. Keeping both engines behind one abstraction also means the local-first choice is a measured decision that can be re-tested at any time, rather than a constraint of the design.

7.6 Why this delivery stack

The OCR core is deliberately interface-agnostic: the web backend, the prototype and the command line all call the same pipeline object. The web platform is built on **React** (Vite) for the interface, **FastAPI** for the API — native multipart upload handling and a natural fit for the Python OCR core — and **MongoDB** for storage, whose document model matches transcription records with heterogeneous metadata better than a relational schema would. Authentication uses JWT with PBKDF2-SHA256 password hashing.

8 Delivered Interfaces

Three interfaces share the same OCR core:

- **Web platform** (primary deliverable) — an EY-branded single-page application: authentication, drag-and-drop upload of images and PDFs, engine and language selection, live progress indication, right-to-left rendering for Arabic-heavy results, per-run metrics (engine, languages, duration, token usage), copy and download, and a permanent searchable per-user history persisted in MongoDB.
- **Streamlit prototype** — a single-file interface for quick one-off tests, showing the original and preprocessed images side by side and offering CER/WER evaluation against a pasted ground truth.
- **Command-line interface** — for scripting and benchmarking; it additionally exposes the three quality-control tools of Section 6 (verification pass, two-model comparison, format validation).

9 Project Structure

The breakdown below reflects the separation between the OCR engines, the processing chain, evaluation and delivery.

Component	Responsibility
Engine layer	Common abstraction over the OCR engines: local implementation (Ollama), cloud implementation (Groq), and the optional classical baseline (Tesseract).
Preprocessing	Image normalisation: denoising, deskew, contrast, resolution capping, PDF rasterisation.
Processing chain	Ingestion, preparation, engine invocation, language detection, Markdown structuring.
Configuration	Model selection, inference parameters (including `num_ctx`), choice of default engine and second-opinion engine.
Quality control	Verification pass, two-model comparison, format validation, CER/WER evaluation.
Web backend	REST API, authentication, per-user document persistence.
Web frontend	Single-page application: upload, engine selection, result viewing, history.
Outputs	Markdown transcriptions produced, kept locally.

Table 4: Functional breakdown of the project

This breakdown makes it possible, in particular, to swap engines — local or cloud — without touching the processing chain, and to add an interface without touching either.

10 Performance

Measured on the target laptop (RTX 4050, 6 GB VRAM) with the default local engine at 300 DPI and `num_ctx` = 8192:

Workload	Time
Printed page	25–29 s
Dense handwritten page	32–47 s
Three-page printed PDF	86 s (measured)
Ten-page form	about 5 minutes
First page after the service is idle	+10 s, one-time (model loads into VRAM)
Verification pass	about 2× (two model calls per page)
Two-model comparison	2.5–3× (runs both the 4B and the 8B)
Format validation	negligible (no model call)

Table 5: Measured processing times

Rule of thumb: **time ≈ pages × 30 s**, plus a one-time 10 s on the first scan after a reboot. Two parameters trade speed against completeness: the context window — the dominant lever, see Section 4.5 — and the PDF rasterisation resolution, where lowering 300 to 250 DPI saves about 20 s per page but can drop the finest print. Both defaults deliberately favour completeness over speed.

11 Development Planning

Phase	Title	Content
1	Study and framing	Analysis of the multilingual requirement, assessment of the limits of classical OCR, identification of the confidentiality and hardware constraints.
2	Model selection	Review of open VLMs, selection of the Qwen3-VL family for its multilingual and document-handling performance.
3	Local prototyping	Bringing Ollama into service, first transcription trials, identification of the *thinking*-mode problem and switch to the `-instruct` variants.
4	Optimisation	Context-window tuning (`num_ctx` = 8192), speed tripled, verification of GPU residency.
5	Comparative evaluation	4B versus 8B on the reference handwritten form; measurement of completeness and timings.
6	Cloud integration	Addition of the Groq / Llama 4 Scout engine as a benchmark and alternative behind the shared engine abstraction.
7	Multi-page and preprocessing	PDF rasterisation at 300 DPI, page-by-page transcription, OpenCV preprocessing chain.
8	Quality control	Verification pass, two-model completeness comparison and format validation of constrained fields.
9	Web platform	REST API, authentication and per-user persistence; branded single-page interface with upload, results and history.
10	Consolidation	Fixing the default engine, setting up the second opinion, documenting the known limitations.

Table 6: Development phases

12 Conclusion

The system uses **Qwen3-VL-4B-Instruct**, an open vision-language model served locally via **Ollama**, as its default OCR engine.

The choice rests on three converging arguments: its multilingual performance, notably in Arabic; its ability to handle documents, tables and handwriting; and — critically — the fact that it runs **fully offline on a 6 GB laptop GPU**, keeping confidential documents on the device.

The 4B size was selected over the 8B after **direct measurement**, which showed it to be both faster and more complete on the target hardware. A cloud engine (Groq / Llama 4 Scout) is retained behind the same abstraction as a higher-capacity alternative and as a benchmark against which the local-first choice can be re-verified.

Around that engine, the project delivers a complete chain rather than a single call: image preprocessing, page-by-page PDF handling, structured Markdown output, three quality-control tools that address the dominant failure mode — omission — from different angles, and a multi-user web platform through which the whole thing is actually usable. What none of it removes is the need for human proofreading of a critical document; the system is built to make that review short, not to make it unnecessary.